Spring Boot internals

How Spring Boot actually works under the hood: startup, the IoC container, auto-configuration, dependency injection, the bean lifecycle, configuration, profiles, starters, embedded servers, and Actuator. The tone is how I would talk each one through with a teammate; the investigation and deployment questions are fully spoken.

1. What is Spring Boot, why was it introduced, and what does it solve?

Spring Boot is an **extension of the Spring Framework** that simplifies building production-ready Spring applications. It reduces boilerplate configuration by providing **auto-configuration, embedded servers, and opinionated defaults**.

Why was it introduced? Traditional Spring applications required significant manual configuration:

- XML or Java configuration.
- Manual dependency management.
- External web server setup (Tomcat, Jetty, and so on).
- Complex project setup.

Spring Boot was introduced to reduce this complexity and let developers build and deploy applications faster.

What problems does it solve?

- Eliminates most boilerplate configuration.
- Automatically configures the application based on the classpath.
- Provides embedded servers (Tomcat, Jetty, Undertow), so no external server deployment is required.
- Offers starter dependencies (for example, `spring-boot-starter-web`) to simplify dependency management.
- Includes production-ready features like health checks, metrics, and externalised configuration.

2. Spring Framework v/s Spring Boot.

	Spring Framework	**Spring Boot**
Configuration	Manual (XML or Java config)	Auto-configured with defaults
Dependencies	Pick and version each yourself	Starters bundle compatible versions
Server	Deploy a war to an external server	Embedded server, runnable jar
Setup effort	High	Low

The one-liner: Spring gives you the **container and features**; Spring Boot gives you a **fast, opinionated way to assemble and run** a Spring app.

3. What are the advantages and disadvantages?

Advantages: fast setup, auto-configuration, starters, embedded server, production-ready features (Actuator), and it is the default for microservices. **Disadvantages**: the **auto-configuration can feel like magic** (hard to know why a bean exists until you learn to read the report), the **fat jar is large**, and if you fight the conventions it

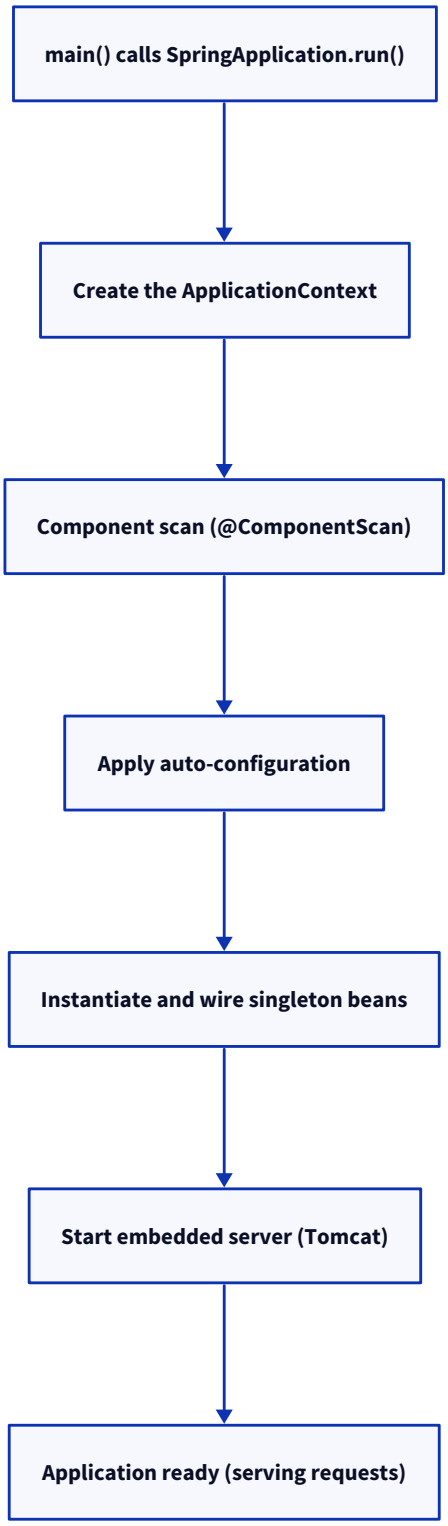
can be more work than plain Spring. So the trade is speed and defaults for a bit of "what is happening under the hood" opacity.

4. Why is Spring Boot so popular for microservices, and what are its key features?

Because a microservice wants to be **small, self-contained, and quick to spin up**, and Spring Boot fits that exactly: each service is a **standalone runnable jar with its own embedded server**, no shared app server, easy to containerise, with **Actuator** giving health and metrics out of the box for orchestration. The **key features** are the same list: auto-configuration, starters, embedded server, Actuator, externalised configuration, and profiles. Put together, you get a service you can build, ship in a container, and monitor with almost no ceremony.

5. Explain the architecture and what happens when a Spring Boot app starts.

The entry point is a `main()` that calls **`SpringApplication.run()`**, and that one call does a lot. `SpringApplication` **creates the ApplicationContext** (the IoC container), **scans for components**, applies **auto-configuration**, **instantiates and wires the singleton beans** (running their lifecycle callbacks), **starts the embedded server**, and finally publishes an **application-ready event**. So `main()` is just the trigger; `SpringApplication.run()` is where the container is built and the app comes alive.



Follow-ups. *What does* `main()` *actually do?* Almost nothing itself, it hands off to `SpringApplication.run(App.class, args)`. *When are* `@PostConstruct` *hooks run?* During bean initialisation, before the app is marked ready.

6. IoC container, ApplicationContext, and BeanFactory.

The **IoC (Inversion of Control) container** is the core of the Spring Framework. It **creates, configures, and manages the lifecycle of beans, and injects dependencies automatically** instead of requiring developers to create and wire objects manually. That inversion, the container doing the creation and wiring rather than your code, is the whole idea. There are two container interfaces. `BeanFactory` is the basic one (lazy, minimal). `ApplicationContext` extends it and adds what you actually want: **eager singleton initialisation**, event publishing, internationalisation, and AOP integration.

	BeanFactory	ApplicationContext
Level	Basic container	Full-featured (extends BeanFactory)
Bean creation	Lazy	Eager for singletons
Extras	None	Events, i18n, AOP, resource loading

In practice you always use **ApplicationContext**, `BeanFactory` is the underlying abstraction.

7. What is @SpringBootApplication, and which annotations does it combine?

`@SpringBootApplication` is a **convenience annotation that marks the main Spring Boot application class**. It enables **auto-configuration, component scanning, and Java-based configuration** with a single annotation.

It combines these three annotations:

- `@Configuration` (via `@SpringBootConfiguration`): marks the class as a source of bean definitions.
- `@EnableAutoConfiguration`: turns on auto-configuration.
- `@ComponentScan`: scans for your components.

You **can** replace it with those three written out separately, it is purely shorthand. Knowing what it expands to matters because it explains the next two questions (scanning and auto-config).

8. How does component scanning work, and where should the main class live?

`@ComponentScan` automatically scans the **package of the main application class and all its sub-packages** for Spring-managed components (such as `@Component`, `@Service`, `@Repository`, and `@Controller`) and **registers them as beans in the IoC container**.

Where should the main class live? The main class annotated with `@SpringBootApplication` should be placed in the **root package** of your application. This ensures that component scanning covers all sub-packages automatically. Put it too deep and half your components silently do not get scanned, a classic "why is my bean null" bug.

Follow-ups. *Bean not being picked up?* Check it is in a package under the main class. *Scan a package outside the main class's tree?* Add it explicitly with `@ComponentScan(basePackages = ...)`.

9. How does auto-configuration work, and how do you exclude one?

Auto-configuration **automatically configures Spring beans based on the dependencies present on the classpath, the existing beans, and the application properties**. It is enabled by `@EnableAutoConfiguration` (included in `@SpringBootApplication`). Under the hood it makes Spring read a list of auto-configuration

classes (from `META-INF/spring/...AutoConfiguration.imports`), each guarded by `@Conditional` annotations: `@ConditionalOnClass` (this library is present), `@ConditionalOnMissingBean` (you have not defined your own), and so on. So if H2 is on the classpath and you defined no `DataSource`, it makes one; define your own and it **backs off**. To **exclude** an auto-configuration you use `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)` or the `spring.autoconfigure.exclude` property.

Follow-ups. *How do you see what auto-configured?* Run with `--debug` for the auto-configuration report (what matched and why). *How does it back off for your bean?* `@ConditionalOnMissingBean`.

10. What is dependency injection, why does it matter, and how does Spring resolve it?

Dependency Injection (DI) is a **design pattern where an object's dependencies are provided by the Spring IoC container instead of the object creating them**. This promotes loose coupling, easier testing, and better maintainability.

Why does DI matter?

- Reduces tight coupling between classes.
- Makes unit testing easier by allowing mock dependencies.
- Improves maintainability and flexibility.
- Lets the IoC container manage object creation and lifecycle.

How does Spring resolve dependencies?

- Spring scans and creates the beans.
- When a bean depends on another bean, Spring looks for a matching bean by type.
- If exactly one matching bean exists, it injects it.
- If multiple beans of the same type exist, you resolve the ambiguity using `@Primary` or `@Qualifier`.
- If no matching bean exists, Spring throws a `NoSuchBeanDefinitionException`.

Types of dependency injection.

1. **Constructor injection (recommended)**: this makes the dependency mandatory, because the object cannot be created unless all required dependencies are provided.
2. **Setter injection**: for optional dependencies.
3. **Field injection**: generally discouraged in production, because it makes testing harder and hides dependencies.

11. Constructor v/s setter v/s field injection, and can Spring inject final fields?

Spring supports **constructor**, **setter**, and **field** injection. Constructor injection is the recommended approach because it makes dependencies mandatory and supports immutable (`final`) fields.

Comparison.

Feature	Constructor injection	Setter injection	Field injection
Dependencies mandatory	Yes	No	No
Supports `final` fields	Yes	No	No
Immutable object	Yes	No	No
Easy to unit test	Yes	Yes	No
Recommended	Yes	For optional dependencies	No

Examples.

1. Constructor injection (recommended).

```
@Service
class UserService {
    private final EmailService emailService;
    public UserService(EmailService emailService) {
        this.emailService = emailService;
    }
}
```

2. Setter injection.

```
@Service
class UserService {
    private EmailService emailService;
    @Autowired
    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }
}
```

Use this when a dependency is **optional** or may need to be changed after object creation.

3. Field injection (not recommended).

```
@Service
class UserService {
    @Autowired
    private EmailService emailService;
}
```

Although it works, it hides dependencies, makes unit testing difficult, and does not support immutable objects.

Can Spring inject `final` fields? Yes, but only through constructor injection.

```
@Service
class UserService {
    private final EmailService emailService;
    public UserService(EmailService emailService) {
        this.emailService = emailService;
    }
}
```

Spring injects the dependency by calling the constructor, where the `final` field is initialised. It **cannot** inject a `final` field through field or setter injection, because a `final` field must be assigned during object construction and cannot be reassigned afterward.

```
@Service
class UserService {
    @Autowired
    private final EmailService emailService; // not supported
}
```

Follow-up: which injection type should you use?

- **Constructor injection**: the default choice for required dependencies.
- **Setter injection**: for optional dependencies.
- **Field injection**: avoid in production code; it is mainly seen in older codebases or quick prototypes.

12. What happens if multiple beans of the same type exist (@Qualifier and @Primary)?

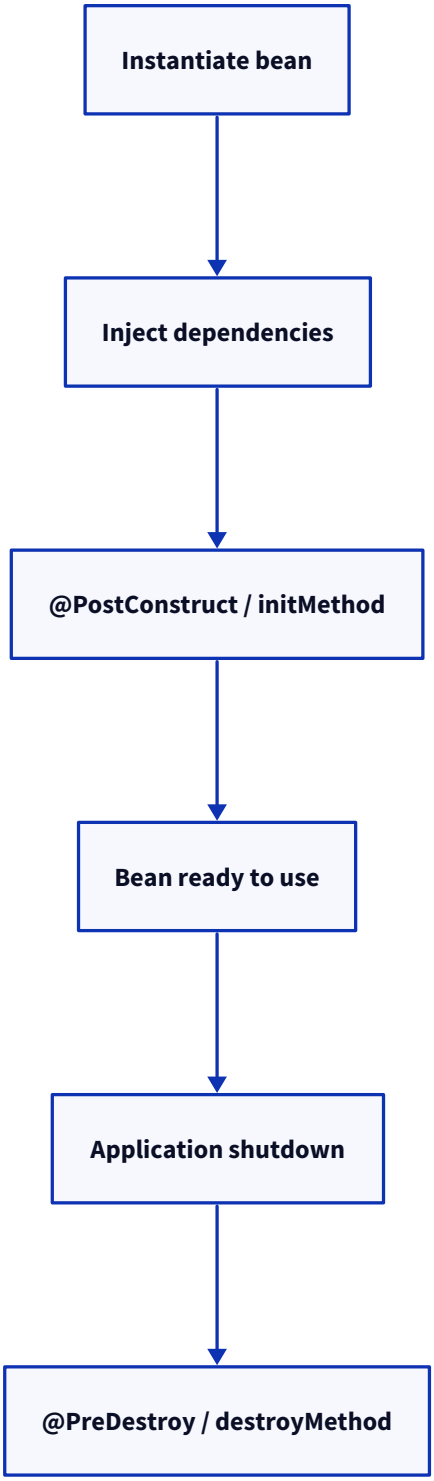
By default Spring cannot decide and throws a `NoUniqueBeanDefinitionException`. You disambiguate two ways: `@Primary` marks one bean as the default choice when several match, and `@Qualifier("name")` picks a specific one at the injection point. Use `@Primary` for the usual default and `@Qualifier` when a particular consumer needs a specific implementation.

13. What is a Spring bean, and what is its lifecycle?

A Spring bean is an **object that is created, configured, and managed by the Spring IoC container**. Spring is responsible for its lifecycle, dependency injection, and destruction.

Bean lifecycle.

- Spring creates the bean.
- Spring injects its dependencies.
- Spring calls the initialisation callbacks (`@PostConstruct` , `InitializingBean` , or a custom `initMethod`).
- The bean is ready for use.
- When the application context shuts down, Spring calls the destruction callbacks (`@PreDestroy` , `DisposableBean` , or a custom `destroyMethod`) for singleton beans.



14. What are the bean scopes?

Scope decides how many instances the container makes and how long they live:

Scope	Lifetime
singleton	**Default**, one instance per container
prototype	A new instance every time it is requested
request	One per HTTP request (web)
session	One per HTTP session (web)
application / websocket	One per servlet context / WebSocket session

You can also define **custom scopes**. The one that trips people: **singleton is per Spring container**, not per JVM, and it is shared across all threads.

15. Lazy v/s eager initialisation, and @Lazy.

By default the container **eagerly creates singleton beans at startup**, which surfaces wiring problems immediately (fail fast) but makes startup slower. `@Lazy` defers a bean's creation **until it is first used**, which speeds startup and avoids building things you might not need, at the cost of moving any failure to first use. So eager is the safe default; reach for `@Lazy` for genuinely expensive or rarely-used beans.

16. How does Spring manage singleton beans, and are they thread-safe?

Spring keeps **one instance per container** and hands the same object to everyone who injects it. And here is the critical point: **a singleton bean is NOT automatically thread-safe**. Because one instance is shared across all request threads, if it holds **mutable instance state**, you have a race. The rule is to keep singleton beans **stateless** (or make any shared state thread-safe). Most service and repository beans are naturally stateless, which is why this usually just works, until someone adds a mutable field.

Follow-ups. *So how do I hold per-request state?* Local variables (on the stack, per thread), or a request-scoped bean, not a field on the singleton.

17. application.properties v/s application.yml.

Both `application.properties` and `application.yml` are configuration files used by Spring Boot. They serve the same purpose; the difference is only the file format.

Comparison.

Feature	`application.properties`	`application.yml`
Format	Key-value pairs	YAML (hierarchical)
Readability	Better for small configurations	Better for complex, nested configurations
Nesting	Repeated keys	Natural indentation
Comments	`#`	`#`
Spring support	Yes	Yes

Example.

application.properties

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update
```

application.yml

```yaml
server:
  port: 8080
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: secret
  jpa:
    hibernate:
      ddl-auto: update
```

Which one should you use?

- **`application.properties`** : simpler for small projects, and easier to edit one property at a time.
- **`application.yml`** : better for large projects with deeply nested configurations, more concise and easier to read for hierarchical settings.

18. How does Spring Boot load configuration, and what is the precedence order?

Spring Boot pulls configuration from **many sources and layers them**, with more specific sources overriding general ones. Roughly high to low precedence: **command-line arguments** > **environment variables / system properties** > **profile-specific files** (`application-{profile}.properties`) > **`application.properties`** / **`.yml`** > **defaults in code**. So you can bake defaults into `application.properties` , override per environment with a profile file, and override again at deploy time with an env var or a command-line flag, without touching the jar. That layering is what "externalised configuration" means.

Follow-ups. *Why does a command-line flag win?* It is the most specific, deploy-time override. *Where do secrets fit?* As env vars or a secrets manager, never committed to the properties file (see Q21).

19. @Value v/s @ConfigurationProperties.

Both `@Value` and `@ConfigurationProperties` are used to read configuration values in Spring Boot. `@Value` is best for a few individual properties, while `@ConfigurationProperties` is designed for binding a group of related properties to a POJO.

Comparison.

Feature	`@Value`	`@ConfigurationProperties`
Best for	Single or few properties	Group of related properties
Nested properties	Difficult	Excellent support
Type-safe binding	Limited	Yes
Validation (`@Validated`)	No	Yes
Maintainability	Lower for many properties	Higher
Recommended	Small configurations	Large / complex configurations

Follow-ups. *Validate config?* Put `@Validated` on the `@ConfigurationProperties` class and use bean-validation annotations, so a bad config fails at startup. *Why not `@Value` everywhere?* It does not scale, no grouping, no type safety. *Relaxed binding?* `@ConfigurationProperties` binds `my-prop` , `my.prop` , `myProp` , and `MY_PROP` to the same field, which is why an env var in caps maps cleanly to a property.

20. What are Spring profiles, and how do you use them?

Spring profiles let you load **different beans and configuration for different environments**, such as development, testing, and production. Only the beans and properties for the **active profile** are loaded.

Why use profiles? Different environments often need different configuration:

- Development: local database, debug logging.
- Testing: in-memory database.
- Production: production database, optimised logging.

Profiles let you switch between these **without changing the code**. In practice you mark environment-specific beans with `@Profile("dev")` , put environment-specific settings in `application-{profile}.properties` , and activate a profile with `spring.profiles.active` .

Follow-ups. *Can multiple profiles be active?* Yes, they layer, comma-separated. *What happens if no profile is active?* Spring uses the default profile and loads only the common configuration from `application.properties` (or `application.yml`). *Can you activate a profile using an environment variable?* Yes, with `SPRING_PROFILES_ACTIVE` (or the `spring.profiles.active` property or a command-line flag).

21. Secrets management and configuration best practices.

The main rule: **never commit secrets** (passwords, API keys) to `application.properties` in the repo. Instead inject them as **environment variables** or pull them from a **secrets manager / vault** at runtime. Beyond that, the practices I would hold to: **externalise anything environment-specific** (do not hardcode), use `@ConfigurationProperties` **with validation** for grouped config, use **profiles** for per-environment differences, and keep sensible **defaults** in `application.properties` for local dev. *(Adapt to your own setup.)*

22. What are Spring Boot starters?

Spring Boot starters are **pre-configured dependency bundles** that bring together all the libraries needed for a specific feature. They simplify dependency management by providing compatible versions of related dependencies.

Why were starters introduced? Without starters, you had to:

- Find all the required dependencies manually.
- Ensure version compatibility.
- Manage the transitive dependencies yourself.

Starters solve this by giving you a **single dependency**. For example, `spring-boot-starter-web` pulls in Spring MVC, Jackson, and embedded Tomcat at compatible versions; `spring-boot-starter-data-jpa` brings JPA and Hibernate; `spring-boot-starter-test` brings JUnit and Mockito.

23. How does Spring Boot manage dependency versions (the BOM)?

Spring Boot manages dependency versions using a **BOM (Bill of Materials)**. The BOM defines tested and compatible versions of commonly used libraries, so you do not need to specify versions for most Spring-related dependencies.

How does it work? When you use Spring Boot, it imports the `spring-boot-dependencies` BOM (via `spring-boot-starter-parent` or an explicit import). This BOM:

- Defines versions for the Spring modules and many third-party libraries.
- Ensures all the dependencies are compatible with each other.
- Eliminates version conflicts in most cases.

So when you add a dependency you **omit the version** and inherit the managed one. You **can override** a managed version (set a property like `<spring.version>` or specify an explicit version), but you do it carefully, because you are stepping outside the tested combination.

Follow-ups. *Why omit versions?* The BOM guarantees a compatible set. *Risk of overriding?* You break the tested compatibility matrix, so verify it.

24. Embedded servers: what is supported, and how do they differ?

Spring Boot supports **embedded web servers**, so applications run as **standalone JARs** without being deployed to an external application server. The default embedded server is **Apache Tomcat**.

Supported embedded servers.

Server	Servlet stack	Reactive stack	Typical use case
Tomcat	Yes	No	Default choice for most applications
Jetty	Yes	No	Lightweight, customisable
Undertow	Yes	No	High-performance, low memory usage
Netty	No	Yes	Spring WebFlux (reactive applications)

Comparison.

Feature	Tomcat	Jetty	Undertow	Netty
Default in Spring Boot	Yes	No	No	No
Servlet-based	Yes	Yes	Yes	No
Reactive	No	No	No	Yes
Performance	Good	Good	Excellent	Excellent
Memory usage	Moderate	Low	Very low	Very low

How does embedded Tomcat work?

1. Add the web starter (`spring-boot-starter-web`), which brings in embedded Tomcat.
2. Start the application (`main` calls `SpringApplication.run`).
3. Auto-configuration detects Tomcat on the classpath.
4. Tomcat starts in-process and serves your app, so the jar is self-running.

Follow-ups. *Can you run without an embedded server?* Yes, for a non-web app or a WAR deployed to an external container. *When would you pick Undertow?* When footprint and throughput matter and you have measured a benefit.

25. WAR v/s JAR, and fat v/s thin jar.

Spring Boot's default and preferred packaging is an **executable ("fat" or "uber") jar**: it bundles your code, all dependencies, **and** the embedded server, so `java -jar app.jar` runs the whole thing, ideal for containers and microservices. A **WAR** is for the older model of deploying to an **external** servlet container, you use it only when you must deploy into existing infrastructure. A **thin jar** externalises the dependencies (smaller artifact, deps fetched/mounted separately), which some deployment setups prefer, but the fat jar is the norm.

26. How would you package and deploy a Spring Boot app for production?

A Spring Boot application is typically packaged as an **executable JAR** and deployed by running `java -jar`. In production it is commonly **containerised with Docker** and orchestrated using **Kubernetes**, or deployed directly to a VM.

Packaging. Build the application with Maven (`mvn package`), which generates the JAR. The JAR contains:

- Your application code.
- All dependencies.
- The embedded server (Tomcat, or Jetty/Undertow).
- Configuration metadata.

Production deployment options.

1. **Docker (most common):** create a Dockerfile, build the image, and run it.
2. **Kubernetes (most common for microservices):** deploy the Docker image to the cluster.
3. **Virtual machine:** copy the JAR to a Linux server and run it.
4. **Cloud platforms (GCP, AWS, and so on):** deploy the JAR or the container image to a managed service.

Across all of these, keep the config externalised (env vars and the active profile) and inject secrets from the platform rather than baking them into the image. *(Adapt to your own pipeline.)*

27. What is Spring Boot Actuator, and why does it matter?

Spring Boot Actuator provides **production-ready features to monitor and manage** a Spring Boot application. It exposes endpoints for health, metrics, application info, environment, logging, and more.

Why does it matter? In production you need to know:

- Is the application healthy?
- Is the database reachable?
- How much memory and CPU is it using?
- How many HTTP requests are being served?
- Can I change log levels without restarting?

Actuator gives you this operational information.

Common endpoints.

Endpoint	Purpose
`/actuator/health`	Application health status
`/actuator/info`	Application information
`/actuator/metrics`	JVM, HTTP, CPU, memory metrics
`/actuator/env`	Environment properties
`/actuator/beans`	All Spring beans
`/actuator/mappings`	All request mappings
`/actuator/loggers`	View and change log levels
`/actuator/threaddump`	Thread dump
`/actuator/heapdump`	JVM heap dump

Follow-ups. *Are all endpoints exposed by default?* No, only `/health` is exposed over HTTP by default; you opt others in with `management.endpoints.web.exposure.include`, and you secure the sensitive ones (`/env`, `/beans`, `/heapdump`) since they leak detail. *How is Actuator used with monitoring tools?* It integrates with Micrometer, which publishes metrics to systems like Prometheus, Datadog, New Relic, and Grafana (usually via Prometheus). For example, in a Datadog setup Actuator exposes the metrics and Micrometer ships them to Datadog for dashboards and alerts. *Custom health indicator?* Implement `HealthIndicator` to fold a downstream dependency's status into `/health`.

28. Your application fails to start because a bean cannot be created. How would you investigate?

The good news is Spring makes this easy, so I start by **reading the failure message**: Boot prints a clear **"APPLICATION FAILED TO START"** block with a **Description** and an **Action** telling you what went wrong and often how to fix it. From there the usual causes: a **missing bean/dependency** (nothing satisfies a required type), **ambiguous beans** (two of the same type, needs `@Qualifier` / `@Primary`), a **circular dependency** (A needs B needs A), a **missing config property**, or the **port already in use**. If it is auto-configuration related, I

rerun with `--debug` for the auto-configuration report to see what did and did not get configured. So: read Boot's failure block first (it is genuinely helpful), identify which of those cases it is, and fix that. *(Adapt to a real one you have hit.)*

29. If Spring Boot provides auto-configuration, why do developers still write configuration classes?

What they are testing: whether you understand the limits of auto-configuration, that it is a starting point, not the whole story.

Because auto-configuration only knows about **common, generic cases**, it cannot know **your domain**. You still write `@Configuration` classes to **define your own beans** (a domain service, a client for a third-party API), to **customise or override a default** (a `RestTemplate` with your timeouts and interceptors, a custom `ObjectMapper`), to **integrate a library that has no starter**, and to do **conditional wiring** your app needs. So auto-config handles the boilerplate defaults, and your config classes handle everything specific to you. The two work together: auto-config backs off (`@ConditionalOnMissingBean`) precisely so your bean wins.

30. Why does Spring Boot make heavy use of conditional annotations?

What they are testing: whether you grasp that auto-configuration has to be **smart and non-intrusive**, and conditionals are how.

Spring Boot uses conditional annotations to apply configuration **only when specific conditions are met**. This is what enables auto-configuration, keeping applications flexible and reducing unnecessary bean creation.

Without conditional annotations, Spring Boot would create every possible bean, even the ones your application does not need. Conditional annotations let it:

- Configure only the required features.
- Avoid bean conflicts.
- Reduce startup time and resource usage.
- Support plug-and-play modules.

The mechanism is the `@Conditional` family: `@ConditionalOnClass` (only if a library is on the classpath), `@ConditionalOnMissingBean` (only if you have not defined your own), and `@ConditionalOnProperty` (only if a flag is set). That is what lets one set of auto-config classes adapt to thousands of apps and back off the moment you define your own bean.

31. How would you design the package structure for a large Spring Boot application with dozens of modules?

What they are testing: architectural judgment at scale, and whether you favour feature cohesion over layer-splitting.

For a large Spring Boot application, organise packages **by feature (domain) rather than by technical layer**. This keeps related code together, improves modularity, and makes the application easier to maintain and scale.

Layer-based structure (not recommended).

```
src/main/java
└── com.company.app
    ├── controller
    ├── service
    ├── repository
    ├── entity
    ├── dto
    ├── config
    └── util
```

Feature-based structure (recommended).

```
src/main/java
└── com.company.app
    ├── customer
    │   ├── controller
    │   ├── service
    │   ├── repository
    │   ├── entity
    │   └── dto
    │
    ├── order
    │   ├── controller
    │   ├── service
    │   ├── repository
    │   ├── entity
    │   └── dto
    │
    ├── payment
    │   ├── controller
    │   ├── service
    │   ├── repository
    │   └── dto
    │
    ├── common
    │   ├── config
    │   ├── exception
    │   ├── security
    │   └── util
    │
    └── Application.java
```

Keep the main class (`Application.java`) in the root package so component scanning covers every feature, and at genuine "dozens of modules" scale use a multi-module build (Maven or Gradle modules) so the boundaries are enforced by the build, not just convention. *(Adapt to your own codebase.)*

32. What engineering guidelines would you establish for Spring Boot development across a large organisation?

What they are testing: whether you can set standards that keep dozens of teams consistent and out of trouble.

The guidelines I would mandate:

- A **standard project and package structure** (prefer feature-based).
- **Constructor injection only**; avoid field injection.

- **Centralised dependency management** using the Spring Boot BOM.
- **Never commit secrets** (use a vault or environment variables).
- A **consistent error model** across services (standard exception handling and response shape).
- **Actuator health wired to the platform probes and secured**.

The meta-rule is **do not fight the conventions**: Spring Boot is opinionated, and teams that work with its grain move faster than teams that reinvent it. *(Adapt to your own org.)*

33. Looking back, which Spring Boot features had the greatest impact on productivity, and which are most commonly misused?

What they are testing: perspective and honesty, can you weigh the trade-offs of the tools you use daily.

The biggest productivity gains in Spring Boot come from **auto-configuration, starter dependencies, embedded servers, and Actuator**, which eliminate boilerplate and simplify deployment. The most commonly misused features are **field injection, excessive auto-configuration, and poor configuration management**.

Features with the greatest productivity impact.

- **Auto-configuration**: reduces manual configuration.
- **Starter dependencies**: simplify dependency management.
- **Embedded servers**: enable standalone JAR deployment.
- **Actuator**: built-in monitoring and health checks.
- **Externalised configuration**: easy environment-specific configuration using properties, YAML, and profiles.

Commonly misused features.

- **Field injection**: makes testing harder and hides dependencies.
- **Overusing** `@Autowired` : prefer constructor injection.
- **Relying blindly on auto-configuration**: you should understand what Spring Boot configures behind the scenes.
- **Exposing all Actuator endpoints**: can introduce security risks.
- **Using** `ddl-auto=update` **in production**: risks unintended schema changes.
- **Ignoring profiles**: leads to hardcoded environment-specific settings.
- **Using** `@Transactional` **without understanding transaction boundaries**: can cause unexpected behaviour and performance issues.

(This is a judgment question, so a defensible take with reasons is what matters.)

34. How does Spring Boot handle concurrent requests, and is it single- or multi-threaded?

What they are testing: whether you understand the thread-per-request model, which is the root of every Spring Boot concurrency question.

Spring Boot is **multi-threaded**. The embedded server (Tomcat by default) keeps a **pool of worker threads** (around 200 by default), and each incoming request is handled on **its own thread** from that pool, so many requests run **concurrently**. Your controller and service code therefore runs on many threads at once.

This is exactly why bean thread-safety matters: your **singleton beans are shared across all those request threads** (see Q16), so a mutable field on a singleton is a race waiting to happen. The mental model to hold: one shared set of beans, many threads calling into them at the same time.

Follow-ups. *So a slow request blocks others?* No, each is on its own thread, but if all pool threads are busy, new requests queue, hence thread-pool sizing matters. *Where is the pool configured?* `server.tomcat.threads.max` and friends.

35. What are the common concurrency challenges in Spring Boot applications?

What they are testing: whether you can name the real traps that come from the thread-per-request model.

The recurring ones:

- **Mutable state on singleton beans**: the single most common bug, a shared bean with an instance field that request threads race on (keep beans stateless, see Q16).
- **Thread pool exhaustion**: every worker thread is blocked on a slow downstream call (no timeout), so the pool fills and new requests queue or time out. Fix with timeouts, circuit breakers, and bounded pools.
- **Connection pool sizing**: too few database connections for the number of worker threads means threads wait for a connection; too many overwhelms the database.
- **Shared caches and counters**: non-thread-safe structures (a plain `HashMap` cache, a `count++`) corrupt under concurrent access, use `ConcurrentHashMap`, atomics, or proper synchronisation.
- **`@Async` and thread pools**: offloaded work needs its own sized executor, and context (security, request scope, trace ids) does not automatically propagate to the async thread.

The theme is the same: anything **shared across request threads** must be stateless or thread-safe.

36. Your API slows down under heavy traffic. How would you investigate thread-related issues?

What they are testing: a methodical approach to a saturated service, and knowledge of the thread-dump technique.

Investigation steps.

1. **Check application metrics**: CPU usage, memory usage, request latency, throughput, and error rate.
2. **Analyse thread dumps**: look for threads in `BLOCKED`, `WAITING`, or `TIMED_WAITING`; check for deadlocks or excessive synchronisation; identify long-running requests.
3. **Inspect the thread pool**: active thread count, queue size, rejected tasks, and pool exhaustion.
4. **Profile the application**: check for lock contention, identify CPU hotspots, and look for blocking database or external API calls.
5. **Review database and external services**: slow SQL queries, connection pool exhaustion, and high API response times.
6. **Use observability tools**: Spring Boot Actuator, Micrometer, and an APM like Datadog, Prometheus, or Grafana.

Common causes.

- Thread pool exhaustion.
- Blocking I/O.
- Database connection pool exhaustion.
- Deadlocks.
- Excessive synchronisation.
- Slow external service calls.
- Infinite loops or CPU-intensive tasks.

The theme across all of these: confirm pool saturation from the metrics, thread-dump to see what the threads are stuck on, and fix the blocking cause rather than just enlarging the pool.

Follow-up: which tools would you use?

- `jstack` : thread dumps.
- `jcmd Thread.print` : JVM thread information.
- Java Flight Recorder (JFR).
- VisualVM or JConsole.
- Spring Boot Actuator (`/actuator/metrics`).
- Datadog or other APM tools.

37. The 30 Spring annotations you should know.

These are 30 annotations worth knowing thoroughly. It is not just about memorising them; you should know what each one does, when to use it, and the common follow-ups.

Annotation	Definition
`@SpringBootApplication`	Marks the main Spring Boot application class and enables auto-configuration, component scanning, and configuration.
`@Component`	Marks a class as a Spring-managed bean.
`@Service`	Specialised `@Component` for the service layer containing business logic.
`@Repository`	Specialised `@Component` for the persistence layer with automatic exception translation.
`@RestController`	Combines `@Controller` and `@ResponseBody` to create REST APIs that return JSON/XML.
`@Configuration`	Marks a class as a source of Spring bean definitions.
`@Bean`	Declares a bean that will be managed by the Spring IoC container.
`@Autowired`	Automatically injects a required dependency into a bean.
`@Qualifier`	Specifies which bean to inject when multiple beans of the same type exist.
`@Primary`	Marks a bean as the default choice when multiple beans of the same type are available.
`@Value`	Injects a single value from configuration properties or expressions.
`@ConfigurationProperties`	Binds a group of related configuration properties to a Java object.
`@Profile`	Activates a bean or configuration only for specific environments (profiles).
`@Transactional`	Defines the transactional boundary for a method or class.
`@PostConstruct`	Runs a method after dependency injection is complete and before the bean is used.
`@PreDestroy`	Runs a method before the Spring bean is destroyed.
`@RequestMapping`	Maps HTTP requests to controller methods.
`@GetMapping`	Maps HTTP GET requests to a controller method.
`@PostMapping`	Maps HTTP POST requests to a controller method.
`@RequestBody`	Binds the HTTP request body to a Java object.
`@PathVariable`	Extracts a value from the URL path and binds it to a method parameter.
`@RequestParam`	Binds a query parameter or form parameter to a method parameter.
`@ExceptionHandler`	Handles specific exceptions thrown by controller methods.
`@RestControllerAdvice`	Provides global exception handling for all REST controllers.

Annotation	Definition
`@Valid`	Triggers bean validation on a method parameter or object.
`@Entity`	Marks a class as a JPA entity mapped to a database table.
`@Id`	Identifies the primary key of a JPA entity.
`@GeneratedValue`	Specifies how the primary key value should be generated.
`@Scheduled`	Schedules a method to run automatically at fixed intervals or times.
`@Cacheable`	Caches the result of a method to improve performance on subsequent calls.

38. Common mistakes.

- **Field injection instead of constructor injection**, which hides dependencies and cannot be `final` or easily tested.
- **Main class in the wrong (too-deep) package**, so component scanning misses your beans.
- **Hardcoding configuration values** instead of externalising them.
- **Misusing `@Value` for complex configuration** where `@ConfigurationProperties` fits.
- **Creating multiple beans of the same type unintentionally**, causing ambiguity errors.
- **Ignoring profile-specific configuration**, so dev settings leak into prod.
- **Exposing sensitive Actuator endpoints** (`/env` , `/beans` , `/heapdump`) without auth.
- **Assuming singleton beans are thread-safe**, then adding mutable state to them.
- **Overriding managed dependency versions** without checking compatibility.
- **Ignoring the auto-configuration report** (`--debug`) when debugging why a bean exists or does not.